

University / Faculty Name

SmartSafe: Smart Safe with Remote Alert

Complete Project Engineering Report:
From Requirements to Verification

Student Name: _____
Student ID: _____
Course: Microprocessors
Instructor: Dr. Raji
Target MCU: Two ATmega32 microcontrollers
Simulation: Proteus Design Suite
Language: Embedded C

Academic Year 1404-1405

Abstract

This report presents the design and implementation of a smart safe controlled by two ATmega32 microcontrollers. The first microcontroller, referred to as the Master, manages password entry, a servo-based door lock, tamper detection, power-failure detection, event logging, low-power operation, and communication. The second microcontroller, referred to as the Slave, acts as a remote alarm and receives event messages exclusively through USART. It displays the latest security state on a 16-by-2 character LCD and controls the remote green LED, red LED, and audible alarm.

The system operates at 8 MHz. The Master uses a matrix keypad for a four-digit password, internal EEPROM for persistent password and failure-count storage, Timer1 for a 50 Hz servo waveform, Timer2 for periodic scheduling and timeout counters, Timer0 for custom blocking delays, ADC3 for tamper sensing, the analog comparator with the internal bandgap reference for supply monitoring, and the hardware SPI unit for a 25LC040 external EEPROM. USART is configured for 9600 baud, 8 data bits, no parity, and one stop bit. The Slave uses the same baud rate and receives complete line-oriented messages, filters log-dump traffic, and applies event-specific display and alarm behavior.

Particular attention is given to register configuration, timing calculations, interrupt behavior, write-one-to-clear status flags, atomic access to multi-byte data, low-power state transitions, and the division of responsibility between the Master and Slave. The report also documents the Proteus wiring, test procedure, design limitations, and possible future improvements.

Contents

Abstract	i
Abbreviations and Symbols	vii
1 Introduction and Project Objectives	1
2 Development from the Original Specification to the Final Design	3
2.1 Requirement decomposition	3
2.2 Pin-allocation process	3
2.3 Clock and communication baseline	4
2.4 Firmware construction sequence	4
2.5 Corrections incorporated in the final reference design	4
2.6 Final reference configuration	5
3 System Architecture	6
3.1 Overall structure	6
3.2 Firmware modularity	6
4 Hardware Design and Proteus Connections	8
4.1 Clock, supply, reset, and analog reference	8
4.2 Master pin assignment	8
4.3 Slave pin assignment	9
4.4 LCD connections	9
4.5 Keypad	10
4.6 Servo	10
4.7 Tamper sensor	10
4.8 Power-failure simulator	10
4.9 External EEPROM	11
4.10 Slave LEDs and alarm	11
5 Clock Selection and Global Timing Basis	13
6 Register Configuration and Calculations	14
6.1 GPIO direction and pull-ups	14
6.2 JTAG disable	14
6.3 USART configuration	15
6.3.1 Transmit completion	15
6.4 Timer0 blocking delays	16

6.4.1	Millisecond delay	16
6.4.2	Microsecond delay	16
6.4.3	Write-one-to-clear behavior	16
6.5	Timer1 servo PWM	16
6.6	Timer2 scheduler	17
6.7	ADC configuration	17
6.8	Analog comparator configuration	18
6.9	SPI configuration	18
6.10	Internal EEPROM registers	18
6.11	External interrupt and sleep configuration	19
6.12	Watchdog timer	19
7	Master Firmware Operation	20
7.1	Initialization order	20
7.2	Keypad scanning	21
7.3	Password storage and checking	21
7.4	Correct password sequence	21
7.5	Incorrect password and lockout	22
7.6	Tamper detection	22
7.7	Power-failure detection and standby	23
7.7.1	Normal monitoring	23
7.7.2	Confirmed failure	23
7.7.3	Restoration	23
7.7.4	Reset while voltage is low	24
7.8	External event log	24
7.9	Terminal commands	24
7.10	Inactivity sleep	24
7.11	Master main-loop structure	25
8	Slave Firmware Operation	26
8.1	Initialization	26
8.2	USART line receiver	26
8.3	Exact message matching	26
8.4	Temporary and persistent output states	27
8.5	BREACH duration and lockout completion	27
8.6	LCD reliability	28
8.7	Log-dump filtering	28
8.8	Slave main loop	28
9	Selection of Constants	29
10	Proteus Test Procedure and Expected Results	31
10.1	Basic startup test	31
10.2	Correct password	31
10.3	Incorrect password and breach	31
10.4	Tamper test	31
10.5	Power-failure test	32
10.6	Inactivity sleep	32
10.7	Log dump	32

11 Limitations and Possible Improvements	33
12 Conclusion	34
A Master Main-Loop Pseudocode	35
B Slave Main-Loop Pseudocode	37
C Selected Register Setup Listings	38
C.1 USART	38
C.2 Timer1 servo PWM	38
C.3 Timer2 scheduler	38
C.4 ADC3	39
C.5 SPI	39
C.6 Analog comparator	39

List of Tables

4.1	Final Master pin map.	8
4.2	Final Slave pin map.	9
4.3	25LC040 connections.	11
7.1	Master internal EEPROM layout.	21
8.1	Final Master-to-Slave protocol.	26
9.1	Important project constants and their rationale.	29

List of Figures

3.1 Overall system architecture.	6
4.1 Power-monitor voltage divider.	11
7.1 External EEPROM memory allocation.	24
7.2 Simplified Master main-loop flow.	25

Abbreviations and Symbols

ADC	Analog-to-Digital Converter
AVR	Microchip/Atmel 8-bit RISC microcontroller architecture
CTC	Clear Timer on Compare Match
EEPROM	Electrically Erasable Programmable Read-Only Memory
GPIO	General-Purpose Input/Output
ISR	Interrupt Service Routine
LCD	Liquid-Crystal Display
LSB	Least Significant Bit
MCU	Microcontroller Unit
PWM	Pulse-Width Modulation
SNOSE	Not used in this project; included only to avoid confusion with unrelated terminology
SPI	Serial Peripheral Interface
USART	Universal Synchronous/Asynchronous Receiver/-Transmitter
WDT	Watchdog Timer
F_{CPU}	CPU clock frequency, 8 MHz
V_{BG}	Internal bandgap reference voltage, nominally about 1.23 V

Chapter 1

Introduction and Project Objectives

The purpose of the project is to design and simulate a smart safe that combines local access control with remote alarm notification. Two ATmega32 microcontrollers are used. The Master is installed with the safe and performs all security decisions. The Slave is a separate remote alarm unit. Communication between the two units is performed only through USART, as required by the project specification [1].

The principal design objectives are:

1. Store a default four-digit password in the Master internal EEPROM.
2. Accept password digits through a 4-by-4 matrix keypad and display one asterisk per digit.
3. Unlock a servo-operated mechanism for five seconds after a correct password.
4. Count failed attempts persistently and enforce a 30-second lockout after three consecutive failures.
5. Detect a sudden tamper-sensor change using the ADC.
6. Detect simulated supply failure using the analog comparator and internal bandgap reference.
7. Store timestamped events in an external 25LC040 SPI EEPROM.
8. Dump logs to a virtual terminal upon receipt of a command.
9. enter a low-power state after approximately ten seconds of inactivity and wake through INT0.
10. Use the watchdog timer to recover from software lockup.
11. Report events to the Slave, where the remote LCD, LEDs, and alarm are controlled.

A deliberate architectural decision was made to place the visual and audible alarm indicators only on the Slave. The Master LCD is therefore used only for password entry. It shows the text Enter Password: while awake and one asterisk for each accepted digit. Security event messages are displayed on the Slave LCD.

Chapter 2

Development from the Original Specification to the Final Design

2.1 Requirement decomposition

The project began with a functional specification rather than a fixed schematic. The required behavior was divided into eight engineering functions: local password entry, mechanical locking, persistent credential storage, tamper sensing, power-failure sensing, event logging, low-power operation, and remote alarm notification. This decomposition led naturally to a two-controller architecture. The Master performs all sensing and security decisions, while the Slave is deliberately simple and reacts only to complete USART messages.

The specification also imposed an important implementation constraint: prepared delay libraries were not to be used. The timing design was therefore established before the peripheral drivers were written. Timer1 was reserved for the servo waveform, Timer2 was selected as the periodic scheduler, and Timer0 was reserved for polled blocking delays. This prevented later conflicts between the LCD, the servo, and application timeouts.

2.2 Pin-allocation process

The pin map was developed by assigning fixed-function peripherals first. OC1A fixed the servo signal to PD5. USART fixed RXD and TXD to PD0 and PD1. Hardware SPI fixed SS, MOSI, MISO, and SCK to PB4-PB7. INT0 fixed the wake button to PD2. The analog comparator required AIN1/PB3 for the monitored-voltage divider when the internal bandgap was connected to the positive comparator input. ADC3/PA3 was then selected for the tamper potentiometer.

The remaining pins were allocated to the LCD and keypad. PORTC was used as the LCD 8-bit data bus. Because the JTAG interface shares PC2-PC5, JTAG is disabled during startup. The keypad was split across PA4-PA7, PD3-PD4, and PB0-PB1 so that it did not conflict with hardware SPI, USART, the servo, the ADC, or the analog comparator.

2.3 Clock and communication baseline

Both ATmega32 devices are configured for an 8 MHz clock. This single choice determines the USART divisor, timer periods, ADC clock, SPI clock, and servo pulse resolution. The USART rate was selected as 9600 baud because it produces a very small divisor error at 8 MHz, is supported directly by the Proteus Virtual Terminal, and is much faster than required for short event messages.

2.4 Firmware construction sequence

The firmware was implemented in layers:

1. GPIO, Timer0 delays, and LCD output were verified first.
2. Timer1 PWM and the servo lock positions were verified independently.
3. Timer2 was added as a common time base for 100 ms tasks and one-second counters.
4. The keypad and internal EEPROM password routines were integrated.
5. ADC tamper sensing and analog-comparator power sensing were added.
6. Hardware SPI and the external event log were integrated.
7. USART event messages and the independent Slave alarm unit were added.
8. Inactivity sleep, comparator standby, watchdog recovery, and terminal log commands were added last because they affect several peripherals simultaneously.

2.5 Corrections incorporated in the final reference design

Several practical issues found during Proteus testing were incorporated into the final reference configuration:

- Timer0 microsecond delay uses no prescaler and OCR0=7. The earlier one-count implementation was not reliable in the simulator.
- LCD R/W is treated as write-only. On the Slave it is tied directly to ground; on the Master it is held low. Conservative execution delays prevent the first character after a clear command from being lost.
- The internal bandgap is allowed to stabilize before the startup comparator state is accepted. A low monitored voltage present during reset is therefore detected immediately.

- Power restoration uses atomic sleep entry so that a comparator transition cannot be lost between the final state check and the SLEEP instruction.
- The Slave ignores lines between the log-dump start and end markers so event names inside log text cannot activate alarms.
- The breach alarm duration is separated from the ordinary five-second alarm duration so it can match the 30-second Master lockout.

2.6 Final reference configuration

Unless a section explicitly discusses an alternative, the report uses the following final reference values:

Item	Final value and reason
CPU clock	8 MHz on both controllers; compatible with all selected timer and communication divisors.
USART	9600 baud, asynchronous 8N1, normal speed, UBRR=51.
Timer0 millisecond delay	CTC, prescaler 64, OCR0=124.
Timer0 microsecond delay	CTC, no prescaler, OCR0=7.
Timer1 servo waveform	Fast PWM mode 14, prescaler 8, ICR1=19999, 50 Hz.
Timer2 scheduler	CTC, prescaler 1024, OCR2=77, compare period 9.984 ms.
ADC	AVCC reference, right-adjusted 10-bit result, ADC3, prescaler 64.
SPI	Master mode 0, MSB first, $F_{CPU}/16 = 500$ kHz.
Power comparator	Internal bandgap on positive input, AIN1/PB3 on negative input, interrupt on output toggle, 100 ms software confirmation.
Normal sleep	Power-down, wake by low-level INT0 push button.
Power-failure standby	Idle mode with only analog-comparator interrupt retained.
Watchdog	Approximately 2.1 s timeout using the avr-libc WDT0_2S configuration.

Chapter 3

System Architecture

3.1 Overall structure

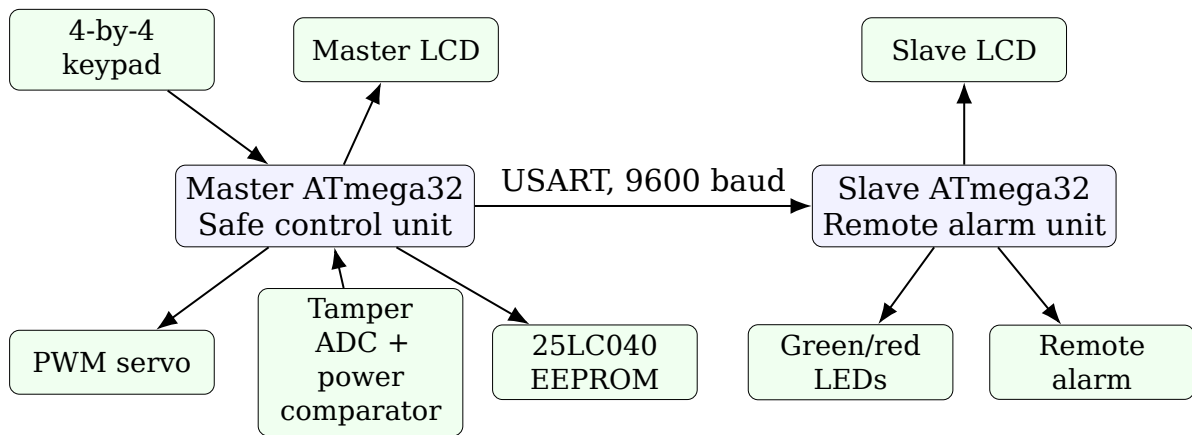


Figure 3.1: Overall system architecture.

The Master performs all sensing and security decisions. The Slave does not read sensors and does not control the safe. It is a message-driven endpoint that updates its outputs according to complete USART lines.

3.2 Firmware modularity

The Master firmware is divided into the following modules:

- `main.c`: initialization order, state coordination, and the main loop;
- `timer.c`: Timer2 scheduler, Timer0 delays, and atomic time access;
- `LCD.c`: 8-bit LCD write-only driver;
- `keypad.c`: mixed-port matrix keypad scanning;
- `servo.c`: Timer1 PWM generation;
- `adc.c`: ADC initialization and blocking conversions;

- `tamper.c`: tamper baseline, threshold decision, and rearming;
- `power.c`: comparator filtering and power-state transitions;
- `spi_eeprom.c`: hardware SPI and 25LC040 byte access;
- `logger.c`: circular event-log management;
- `password.c`: internal EEPROM password and failed-attempt management;
- `usart.c`: line transmission and command reception.

The Slave firmware is smaller. It contains LCD, USART, Timer0/Timer2, output-state handling, and a line-oriented message dispatcher.

Chapter 4

Hardware Design and Proteus Connections

4.1 Clock, supply, reset, and analog reference

Both ATmega32 models are configured for an 8 MHz clock in their Proteus properties. The simulation does not require an external crystal when the clock property is set directly. In physical hardware, either the internal oscillator must be configured for 8 MHz through fuse settings or an external 8 MHz crystal with suitable load capacitors may be used.

The following supply connections are required on each ATmega32:

- VCC to 5 V;
- both GND pins to ground;
- AVCC to 5 V;
- AREF to a 100 nF capacitor to ground when AVCC is selected as the ADC reference.

A 10 k Ω pull-up from RESET to 5 V and a push button from RESET to ground are recommended. The button is not required for normal operation but provides a convenient manual restart during simulation.

4.2 Master pin assignment

Table 4.1: Final Master pin map.

Port pin	Alternate function	Project connection
PA0	ADC0	LCD RS
PA1	ADC1	Unused; LCD R/W is tied to ground
PA2	ADC2	LCD Enable
PA3	ADC3	Tamper potentiometer wiper

Port pin	Alternate function	Project connection
PA4-PA7	ADC4-ADC7	Keypad rows A-D
PB0	T0/XCK	Keypad column 3
PB1	T1	Keypad column 4
PB2	AIN0/INT2	Unused in the final comparator connection
PB3	AIN1/OC0	Power-monitor divider midpoint
PB4	SS	25LC040 chip select
PB5	MOSI	25LC040 SI
PB6	MISO	25LC040 SO
PB7	SCK	25LC040 SCK
PC0-PC7	GPIO/JTAG/TOSC	LCD D0-D7
PD0	RXD	Virtual-terminal command input
PD1	TXD	Slave RX and optional terminal monitor
PD2	INT0	Wake button, active low
PD3	INT1	Keypad column 1
PD4	OC1B	Keypad column 2
PD5	OC1A	Servo PWM signal
PD6-PD7	GPIO	Unused or temporary diagnostic pins

4.3 Slave pin assignment

Table 4.2: Final Slave pin map.

Port pin	Alternate function	Project connection
PA0	ADC0	LCD RS
PA2	ADC2	LCD Enable
PA3	ADC3	Alarm-driver control
PC0-PC7	GPIO/JTAG/TOSC	LCD D0-D7
PD0	RXD	Master TXD
PD1	TXD	Unused
PD6	ICP1	Green LED through a series resistor
PD7	OC2	Red LED through a series resistor

4.4 LCD connections

Both LM016L displays are operated in 8-bit mode. D0-D7 are connected in order to PC0-PC7. RS is connected to PA0 and Enable to PA2. R/W is tied directly to ground because the firmware never reads the busy flag. Instead, all required execution delays are generated by Timer0. The contrast pin

VEE is connected to the wiper of a 10 k Ω potentiometer whose end terminals are connected to 5 V and ground. A contrast voltage close to ground normally gives visible characters in Proteus.

JTAG shares several PORTC pins on the ATmega32. Consequently, JTAG is disabled at startup by writing the JTD bit in MCUCSR twice within the required timing window.

4.5 Keypad

The SMALLCALC keypad has rows A-D on the left and columns 1-4 at the bottom. The final wiring is:

A $\rightarrow$ PA4, B $\rightarrow$ PA5, C $\rightarrow$ PA6, D $\rightarrow$ PA7,
1 $\rightarrow$ PD3, 2 $\rightarrow$ PD4, 3 $\rightarrow$ PB0, 4 $\rightarrow$ PB1.

The Proteus keypad layout is interpreted as:

7	8	9	/
4	5	6	×
1	2	3	−
C	0	E	+

Only decimal digits and C are used. Four digits cause automatic password evaluation; C clears the current entry.

4.6 Servo

The PWM input of the MOTOR-PWMSERVO component is connected to PD5/OC1A. Its power pins are connected to 5 V and ground. The model is configured for a 20 ms frame, with approximately 1 ms corresponding to the locked position and 1.5 ms corresponding to the unlocked position.

4.7 Tamper sensor

A POT-HG potentiometer simulates displacement or vibration. Its end terminals are connected to 5 V and ground, and its wiper is connected to PA3/ADC3. A sudden or accumulated displacement from the stable reference is interpreted as tampering.

4.8 Power-failure simulator

The Master remains physically powered during the power-failure simulation. A separate variable monitored voltage is scaled and applied to PB3/AIN1:

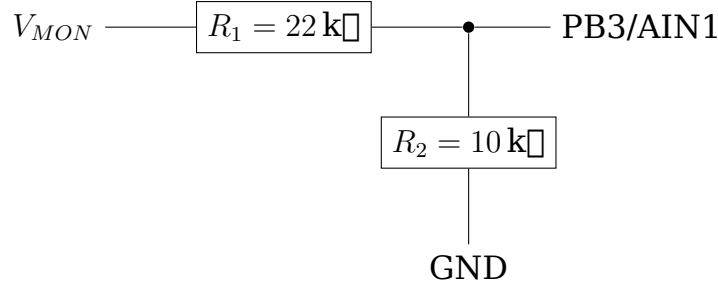


Figure 4.1: Power-monitor voltage divider.

The divider output is

$$V_{AIN1} = V_{MON} \frac{R_2}{R_1 + R_2} = V_{MON} \frac{10}{22 + 10} = 0.3125 V_{MON}. \quad (4.1)$$

At a nominal internal bandgap of 1.23 V, the monitored-voltage trip point is

$$V_{TRIP} = 1.23 \frac{22 + 10}{10} \approx 3.94 \text{ V}. \quad (4.2)$$

At 5 V, the divider current is only

$$I = \frac{5}{22\,000 + 10\,000} \approx 156 \mu\text{A}. \quad (4.3)$$

4.9 External EEPROM

The 25LC040 is connected to the Master hardware SPI pins. WP and HOLD are tied high, VCC is connected to 5 V, and VSS is grounded. A local 100 nF decoupling capacitor is placed between VCC and ground. The connection is summarized in [table 4.3](#).

Table 4.3: 25LC040 connections.

25LC040 signal	Master pin	Condition
CS	PB4/SS	Active low
SI	PB5/MOSI	Master to EEPROM
SO	PB6/MISO	EEPROM to Master
SCK	PB7/SCK	SPI clock
WP	VCC	Write protection disabled
HOLD	VCC	Communication not paused
VCC	5 V	Supply
VSS	GND	Common reference

4.10 Slave LEDs and alarm

PD6 drives the green LED and PD7 drives the red LED through individual 220 Ω to 470 Ω series resistors. PA3 controls the alarm. A passive low-impedance speaker must not be connected directly to the MCU pin. In physical hardware, PA3 should drive an NPN transistor through approximately

1 k Ω , with the buzzer connected between 5 V and the transistor collector. The emitter is grounded. An active buzzer is compatible with the present high/low control strategy. If a passive speaker is used, an audio-frequency waveform must additionally be generated.

Chapter 5

Clock Selection and Global Timing Basis

The entire project is based on

$$F_{CPU} = 8 \text{ MHz.} \quad (5.1)$$

This frequency was selected because it is directly supported by the ATmega32, provides adequate processing speed, permits a low USART baud error at 9600 baud, produces convenient integer timer values, and remains easy to configure in Proteus.

The C macro `F_CPU` does not set the physical clock. It documents the assumed clock for calculations and delay code. Therefore, both Proteus microcontrollers must also have their clock property set to 8 MHz.

Chapter 6

Register Configuration and Calculations

6.1 GPIO direction and pull-ups

Each AVR port has three main registers:

- DDRx: selects input (0) or output (1);
- PORTx: writes an output value or enables an input pull-up;
- PINx: reads the physical pin state.

For example, the tamper input is configured using

```
1 DDRA  &= ~(1U << PA3);    /* input */
2 PORTA &= ~(1U << PA3);    /* internal pull-up disabled */
```

The keypad columns use the opposite PORTx setting: they remain inputs but have PORTx set high to enable internal pull-ups.

6.2 JTAG disable

Because PC2–PC5 are JTAG pins, the JTD bit in MCUCSR is written twice. Interrupts are temporarily disabled so the two writes occur without delay:

```
1 uint8_t sreg = SREG;
2 cli();
3 uint8_t value = MCUCSR | (1U << JTD);
4 MCUCSR = value;
5 MCUCSR = value;
6 SREG = sreg;
```

This allows all eight PORTC pins to operate as LCD data outputs.

6.3 USART configuration

The asynchronous normal-speed baud-rate equation is

$$UBRR = \frac{F_{CPU}}{16 \times Baud} - 1. \quad (6.1)$$

For 8 MHz and 9600 baud,

$$UBRR = \frac{8\,000\,000}{16 \times 9600} - 1 = 51.0833. \quad (6.2)$$

The nearest integer is 51. The actual baud rate is

$$Baud_{actual} = \frac{8\,000\,000}{16(51 + 1)} \approx 9615.38 \text{ baud}, \quad (6.3)$$

which is approximately 0.16 percent high and is well within the tolerance of this short Proteus connection.

The Master USART initialization is:

```

1 UBRRH = (uint8_t)(51U >> 8);
2 UBRRL = (uint8_t)51U;
3 UCSRB = (1U << RXEN) | (1U << TXEN);
4 UCSRC = (1U << URSEL) |
5         (1U << UCSZ1) | (1U << UCSZ0);

```

- RXEN=1 enables reception of terminal commands.
- TXEN=1 enables transmission to the Slave and terminal.
- URSEL=1 selects UCSRC rather than UBRRH on the ATmega32 shared address.
- UCSZ1:UCSZ0=11 selects 8-bit characters.
- Parity bits remain zero, selecting no parity.
- USBS=0 selects one stop bit.

The resulting frame is 8N1. Each transmitted character contains one start bit, eight data bits, and one stop bit. Therefore, one byte takes approximately

$$\frac{10}{9600} \approx 1.042 \text{ ms}. \quad (6.4)$$

The Slave enables only RXEN, because it does not transmit acknowledgements.

6.3.1 Transmit completion

The ordinary transmit routine waits for UDRE, indicating that the data register can accept another byte. Before entering power-failure standby, the code must instead wait for TXC, which indicates that the final stop bit has physically left TXD. Since TXC is write-one-to-clear, the code clears it before transmitting and polls it afterward.

6.4 Timer0 blocking delays

Prepared delay libraries are not used. Timer0 is temporarily reconfigured for LCD timing, and its previous register values are restored afterward.

6.4.1 Millisecond delay

Timer0 is placed in CTC mode using $WGM01=1$ and $WGM00=0$. A prescaler of 64 is selected with $CS01=1$ and $CS00=1$. The timer frequency is

$$\frac{8\,000\,000}{64} = 125\text{ kHz}, \quad (6.5)$$

so one count lasts $8\text{ }\mu\text{s}$. A compare value of $OCR0=124$ produces 125 counts from 0 through 124:

$$125 \times 8\text{ }\mu\text{s} = 1\text{ ms}. \quad (6.6)$$

6.4.2 Microsecond delay

For the corrected microsecond delay, Timer0 runs without a prescaler. One count is $0.125\text{ }\mu\text{s}$. Setting $OCR0=7$ gives eight counts:

$$8 \times 0.125\text{ }\mu\text{s} = 1\text{ }\mu\text{s}. \quad (6.7)$$

The C loop adds a small positive overhead, which is acceptable because LCD timing values are minimum requirements.

6.4.3 Write-one-to-clear behavior

$OCF0$ in $TIFR$ is cleared by writing one, not zero:

```
1 TIFR = (1U << OCF0);
```

A read-modify-write form such as $TIFR |= \dots$ is avoided because it could unintentionally clear other set timer flags.

6.5 Timer1 servo PWM

Timer1 operates in Fast PWM mode 14. The waveform-generation bits are

$$WGM13 : WGM12 : WGM11 : WGM10 = 1110. \quad (6.8)$$

This is implemented using $WGM13$ and $WGM12$ in $TCCR1B$, and $WGM11$ in $TCCR1A$. $COM1A1=1$ selects a non-inverting waveform on $OC1A$. A prescaler of 8 is selected using $CS11=1$.

The timer clock is

$$\frac{8\,000\,000}{8} = 1\text{ MHz}, \quad (6.9)$$

which gives one microsecond per timer count. With ICR1=19999, the counter period is 20,000 counts:

$$20\,000 \times 1\,\mu\text{s} = 20\,\text{ms} = \frac{1}{50}\,\text{s}. \quad (6.10)$$

The pulse width is mapped linearly from an angle:

$$OCR1A = 1000 + \frac{\text{angle} \times 1000}{180}. \quad (6.11)$$

Thus 0 degrees gives 1 ms, 90 degrees gives 1.5 ms, and 180 degrees gives 2 ms. The safe uses 0 degrees as locked and 90 degrees as unlocked.

6.6 Timer2 scheduler

Timer2 supplies the regular system tick for both microcontrollers. It is configured in CTC mode with WGM21=1. The prescaler is 1024, selected by CS22:CS21:CS20=111. The timer frequency is

$$\frac{8\,000\,000}{1024} = 7812.5\,\text{Hz}. \quad (6.12)$$

One count lasts 128 μs . Setting OCR2=77 gives 78 counts:

$$78 \times 128\,\mu\text{s} = 9.984\,\text{ms}. \quad (6.13)$$

Ten compare interrupts produce approximately 99.84 ms, and ten of those periods produce approximately 998.4 ms. The small 0.16 percent timing error is consistent with the USART clock error and is negligible for the 1-, 5-, 10-, and 30-second project intervals.

The compare interrupt is enabled by OCIE2 in TIMSK. OCF2 is cleared by writing one to TIFR. The ISR performs only short operations: it advances sub-counters, decrements timeout counters, updates the timestamp, and advances the power-comparator confirmation timer.

6.7 ADC configuration

The ADC uses AVCC as the reference. In ADMUX, REFS0=1 and REFS1=0 select AVCC. ADLAR=0 keeps the 10-bit result right-adjusted. The channel field MUX4:0 is set to 3 for ADC3/PA3.

The ADC is enabled using ADEN=1 in ADCSRA. The prescaler bits ADPS2:ADPS1:ADPS0=110 select division by 64:

$$F_{ADC} = \frac{8\,000\,000}{64} = 125\,\text{kHz}. \quad (6.14)$$

This lies in the recommended range for full 10-bit accuracy [2]. A conversion is started by setting ADSC; the code waits until hardware clears ADSC. At a 5 V reference, the nominal ADC step is

$$\Delta V = \frac{5}{1024} \approx 4.883\,\text{mV}. \quad (6.15)$$

A tamper threshold of 50 counts therefore corresponds to approximately

$$50 \times 4.883\,\text{mV} \approx 0.244\,\text{V}. \quad (6.16)$$

6.8 Analog comparator configuration

The comparator positive input is connected internally to the bandgap by setting $ACBG=1$ in $ACSR$. The external divider is connected to $AIN1/PB3$, the comparator negative input. The resulting interpretation is:

$$ACO = 1 \iff V_{BG} > V_{AIN1}, \quad (6.17)$$

which is treated as a power failure.

The interrupt is enabled by $ACIE=1$. $ACIS1:ACIS0=00$ selects interrupt on comparator output toggle. ACI is write-one-to-clear. During initialization, the comparator interrupt is disabled, the bandgap is enabled, and a short stabilization delay of approximately 1 ms is applied before reading the initial state. This prevents a reset during low voltage from being misclassified as normal power.

Each comparator transition starts or restarts a 10-tick confirmation interval. Since Timer2 ticks every approximately 10 ms, the candidate state must remain stable for about 100 ms. This software filter suppresses threshold chatter.

6.9 SPI configuration

The Master uses the dedicated SPI hardware. In $SPCR$:

- $SPE=1$ enables SPI;
- $MSTR=1$ selects Master mode;
- $SPR0=1$ and $SPR1=0$ select $F_{CPU}/16$;
- $CPOL=0$ and $CPHA=0$ select SPI mode 0;
- $DORD=0$ sends the most significant bit first.

$SPSR$ has $SPI2X=0$, so the SPI clock is

$$F_{SCK} = \frac{8\,000\,000}{16} = 500 \text{ kHz}. \quad (6.18)$$

This is comfortably below the EEPROM maximum clock rate [3].

A byte transfer writes $SPDR$ and waits for $SPIF$ in $SPSR$. Reading $SPDR$ returns the simultaneously received byte. The 25LC040 commands used are $WREN$ (0x06), $WRITE$ (0x02), $READ$ (0x03), and $RDSR$ (0x05). Because the device has 512 bytes, address bit A8 is encoded in bit 3 of the instruction byte for addresses above 255.

6.10 Internal EEPROM registers

The Master internal EEPROM stores a signature, four password bytes, and the failed-attempt counter. Before a read or write, the code waits until $EEWE=0$ in $EECR$. A read loads $EEAR$, sets $EERE$, and reads $EEDR$. A write loads $EEAR$ and $EEDR$, then performs the timed sequence:

```
1 EECR |= (1U << EEMWE);  
2 EECR |= (1U << EEWE);
```

Interrupts are disabled around these two writes so that the required sequence is not interrupted.

6.11 External interrupt and sleep configuration

The normal inactivity sleep mode is Power-down. INT0 is configured for low-level activation by clearing ISC01 and ISC00 in MCUCR. The pending flag INTF0 in GIFR is cleared by writing one, and INT0 is enabled through INT0 in GICR. The wake button connects PD2 to ground and uses the internal pull-up.

The INT0 ISR disables INT0 immediately because a low-level interrupt remains active while the button is held. INT0 is enabled again only immediately before the next sleep.

Two sleep modes are used:

- **Power-down** after ten seconds of ordinary inactivity. The analog comparator interrupt is disabled, and INT0 is the only intended wake source.
- **Idle** after a confirmed simulated power failure. Timers, USART, SPI, ADC, INT0, LCD, and servo output are disabled manually; the comparator interrupt remains active so restoration can wake the CPU.

6.12 Watchdog timer

The watchdog is enabled with an approximately two-second timeout using `wdt_enable(WDT0_2S)`. The main loops call `wdt_reset()` regularly. The watchdog is disabled before deliberate sleep because the main loop cannot reset it while sleeping. At startup, the `WDRF` flag in `MCUCSR` is checked. If set, the Master sends `WDT_RESET` and logs the event.

Chapter 7

Master Firmware Operation

7.1 Initialization order

The Master initializes its subsystems in a controlled sequence:

1. disable JTAG for PORTC;
2. configure the INT0 pin and keep its interrupt disabled;
3. initialize USART;
4. initialize the LCD;
5. initialize Timer1 and lock the servo;
6. initialize ADC3;
7. initialize hardware SPI;
8. validate or create the external EEPROM log header;
9. validate or create the internal EEPROM password data;
10. sample the current tamper baseline;
11. initialize Timer2;
12. initialize the analog comparator, wait for bandgap stabilization, and sample the startup state;
13. enable global interrupts.

The password prompt is drawn by the first main-loop iteration. This makes it clear that initialization has completed before user interaction begins.

7.2 Keypad scanning

The keypad uses four output rows and four pull-up input columns. A scan proceeds as follows:

1. configure all rows as high outputs;
2. configure all columns as inputs with pull-ups;
3. drive one row low while keeping the others high;
4. read the four columns;
5. if a column is low, identify the corresponding key;
6. return all rows high;
7. report a key only once until it is released.

The static `last_reported` variable implements edge-like behavior. Holding a key does not repeatedly enter the same digit, while pressing and releasing the same digit several times remains valid.

7.3 Password storage and checking

The internal EEPROM layout is shown in [table 7.1](#).

Table 7.1: Master internal EEPROM layout.

Address	Size	Purpose
0x00	1 byte	Signature 0xA5
0x01-0x04	4 bytes	Password characters
0x05	1 byte	Failed-attempt count

If the signature is absent, the default password 1234 is written. The input buffer contains four digits plus a null terminator. Each accepted digit is displayed as an asterisk. After four digits, the buffer is compared byte-by-byte with the EEPROM password.

7.4 Correct password sequence

When the password is correct:

1. reset the persistent failed-attempt count;
2. log `EVT_LOGIN_OK`;
3. send `LOGIN_OK`;
4. move the servo to 90 degrees;

5. set servo_open=1 and load servo_counter=5;
6. block further keypad input and prevent sleep during the five-second interval;
7. return the servo to 0 degrees when the counter reaches zero;
8. clear the entry and restart the inactivity timer.

The green indication is performed by the Slave, not by the Master.

7.5 Incorrect password and lockout

For an incorrect password, the failed-attempt count is incremented in internal EEPROM and EVT_LOGIN_FAIL is logged. Before the third failure, the Master sends FAILURE and applies a one-second keypad hold. At the third failure, it sends BREACH, logs EVT_LOCKOUT, and sets a 30-second lockout counter.

During lockout, keypad input and inactivity sleep are both blocked. When the counter reaches zero, the Master clears the failed-attempt count, clears the input buffer, resets the activity timer, and sends LOCKOUT_END. The Slave uses this message to turn off the 30-second breach indication and return its LCD to the waiting screen.

7.6 Tamper detection

Timer2 sets a 100-ms software flag. The main loop clears the flag and calls the tamper function. A robust implementation uses a stable reference rather than replacing the reference after every sample. This allows several small manual potentiometer movements to accumulate until the total displacement exceeds the threshold.

The recommended state variables are:

- reference_adc: the stable sensor reference;
- armed: whether a new event is allowed;
- stable_samples: consecutive stable readings after an event.

The algorithm is:

1. read ADC3 every approximately 100 ms;
2. calculate the absolute difference from reference_adc;
3. when armed and the difference is greater than 50 counts, return one tamper event;
4. set the current value as the new reference and disarm;
5. require several consecutive readings within a small rearm band before arming again.

When a new tamper event is returned, the Master sends Tamper, logs EVT_TAMPER, starts a five-second UI hold, and resets the inactivity timer. The Master LCD is not replaced by a tamper message.

7.7 Power-failure detection and standby

7.7.1 Normal monitoring

The analog comparator runs continuously, but its ISR runs only when the comparator output changes. A stable normal or failed voltage does not repeatedly generate interrupts. Each toggle starts a 100-ms software confirmation interval. The main loop accepts the new state only if the live comparator still agrees when the interval expires.

7.7.2 Confirmed failure

After a confirmed failure:

1. read an atomic timestamp;
2. send POWER_FAIL and wait for the final stop bit;
3. store EVT_POWER_FAIL in the external EEPROM;
4. turn off the Master LCD;
5. disable the watchdog;
6. stop Timers 0, 1, and 2 and disable timer interrupts;
7. disable INT0, ADC, SPI, and USART;
8. enter Idle sleep with the comparator interrupt active.

Idle mode is used because the simulated Master supply remains present and the comparator must wake the MCU when the monitored voltage returns.

7.7.3 Restoration

When the comparator reports normal voltage, the Master exits the standby loop and reinitializes USART, LCD, servo, ADC, tamper baseline, SPI, and Timer2. It then sends POWER RESTORED, logs EVT_POWER_OK, clears partial password input, and redraws the prompt.

7.7.4 Reset while voltage is low

On startup, the bandgap is allowed to stabilize before the comparator is sampled. If the monitored voltage is already below threshold, `power_event_pending` is set immediately. This ensures that pressing RESET during an existing failure causes the Master to report the failure and return to standby without waiting for another potentiometer transition.

7.8 External event log

The 25LC040 has 512 bytes. Eight bytes are reserved for a header and the remaining 504 bytes store 63 records of eight bytes each.

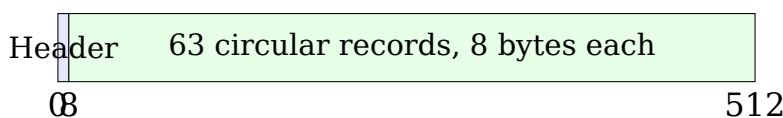


Figure 7.1: External EEPROM memory allocation.

The header contains two magic bytes, the next write index, the valid record count, and four reserved bytes. Each record contains an event code, a 32-bit big-endian timestamp, a 16-bit data field, and an XOR checksum. The write index wraps to zero after record 62. When all records are valid, the oldest record is overwritten.

7.9 Terminal commands

The Master polls its USART receiver. The final commands are:

- D or d: dump the log;
- C or c: clear the log;
- X or x: optional compatibility alias for clear.

Any recognized terminal command resets the inactivity timer before and after the operation. This prevents a long blocking log dump from ending with immediate sleep. The virtual terminal may use a switch on its RX connection so ordinary event messages are not displayed. Its TX line remains connected to Master RX for commands.

7.10 Inactivity sleep

The activity counter increments once per second. It is reset by a key press, terminal command, the start and end of a UI hold, wake-up, and power restoration. Sleep is permitted only when:

- the counter has reached ten seconds;

- lockout is not active;
- the servo is not open;
- no UI hold is active.

The Master logs EVT_SLEEP, turns off the LCD, disables the comparator interrupt, enables low-level INT0, disables the watchdog, and enters Power-down. After the button wakes the MCU, it logs EVT_WAKE, resynchronizes the comparator state, turns the LCD on, and restarts the ten-second activity interval.

7.11 Master main-loop structure

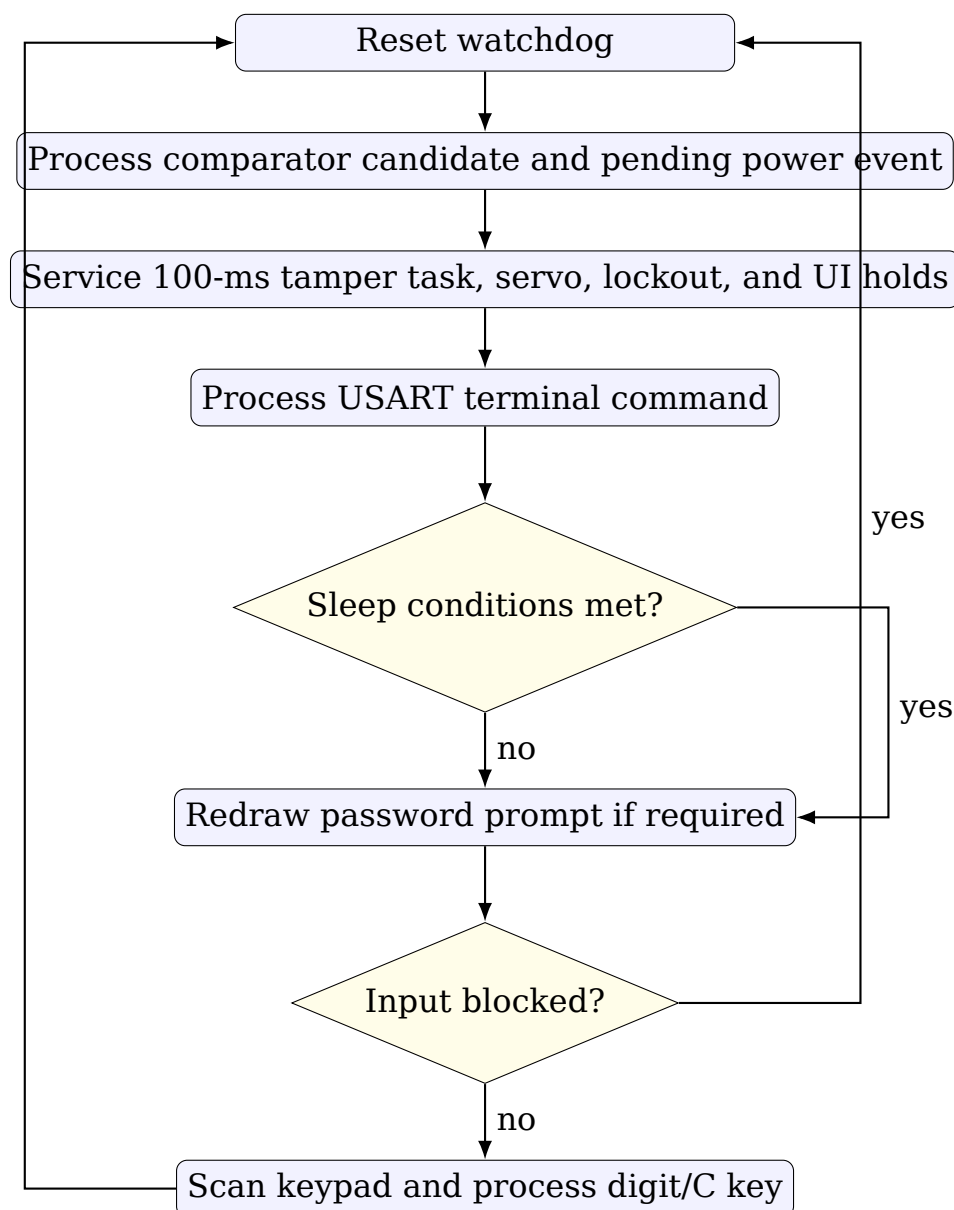


Figure 7.2: Simplified Master main-loop flow.

Chapter 8

Slave Firmware Operation

8.1 Initialization

The Slave disables JTAG, configures PD6 and PD7 as LED outputs, configures PA3 as the alarm output, initializes its receive-only USART, initializes the LCD and Timer2, enables interrupts, and displays:

```
REMOTE ALARM  
WAITING...
```

The watchdog is then enabled with a two-second timeout.

8.2 USART line receiver

The Slave polls RXC. Received bytes are appended to an 80-byte array until carriage return or line feed is received. A null terminator is appended before string comparison. Since the Master sends CR/LF, the CR processes the line and the following LF is ignored because the buffer is empty.

A 79-character maximum prevents overflow. If the line is too long, the entire line is discarded until the next CR or LF. This is safer than processing a truncated line.

8.3 Exact message matching

Messages are compared using strcmp. Exact comparison prevents a log record containing a word such as TAMPER or BREACH from falsely activating the remote alarm.

Table 8.1: Final Master-to-Slave protocol.

Message	Slave LCD	Output action
LOGIN_OK	ACCESS GRANTED / SAFE OPEN	Green LED for 5 s
FAILURE	WRONG PASSWORD / TRY AGAIN	Red LED and alarm for 5 s

Message	Slave LCD	Output action
BREACH	SECURITY BREACH / MASTER LOCKED	Red LED and alarm for 30 s
LOCKOUT_END	REMOTE ALARM / WAITING...	Clear breach indication
Tamper	TAMPER DETECTED / CHECK SAFE	Red LED and alarm for 5 s
POWER_FAIL	POWER FAILURE / MASTER OFFLINE	Persistent red LED and alarm
POWER RE-STORED	POWER RESTORED / SYSTEM NORMAL	Clear persistent alarm
WDT_RESET	MASTER RESTARTED / WATCHDOG RESET	LCD diagnostic only

8.4 Temporary and persistent output states

The Slave distinguishes three independent states:

- a five- or thirty-second temporary red alarm;
- a five-second green success indication;
- a persistent power-failure alarm.

A persistent power alarm has priority and can be cleared only by POWER RESTORED. Ordinary success or failure messages do not override it.

Timer2 decrements alarm_counter and green_counter once per second. The ISR only changes counters. The main loop turns outputs off when the counters reach zero. This keeps the ISR short and avoids calling LCD or complex application functions from interrupt context.

8.5 BREACH duration and lockout completion

The ordinary remote alarm duration is five seconds, but BREACH is intentionally set to thirty seconds to match the Master lockout. A separate constant is used:

```

1 #define REMOTE_ALARM_SEC  5U
2 #define BREACH_ALARM_SEC 30U
3 #define SUCCESS_LED_SEC   5U

```

The Slave also receives LOCKOUT_END, which clears the outputs and restores the waiting screen. This avoids leaving MASTER LOCKED on the LCD after the Master becomes usable.

8.6 LCD reliability

Both Slave LCD control and data signals are write-only. R/W is grounded. The custom Timer0 delays provide a setup delay before Enable rises, an Enable pulse, and a post-write execution delay. Clear-display and return-home commands receive approximately 3 ms of margin; ordinary commands and data writes receive approximately 100 μ s. These conservative values avoid losing the first character after a clear operation in Proteus.

The line-writing routine writes exactly sixteen positions and pads unused positions with spaces. This removes leftover characters from an earlier longer message.

8.7 Log-dump filtering

The Master begins a dump with `=== LOG DUMP ===` and ends with `=== END ===`. The Slave sets a `log_dump_active` flag after the start marker and discards all lines until the end marker. Consequently, the virtual terminal can display the logs while the Slave LCD and indicators remain unchanged.

8.8 Slave main loop

The Slave main loop performs four tasks:

1. reset the watchdog;
2. service expired LED and alarm counters;
3. poll the USART receiver;
4. assemble and dispatch complete lines.

No blocking wait is used for incoming communication, so output timeouts continue to be serviced even when no messages are arriving.

Chapter 9

Selection of Constants

Table 9.1: Important project constants and their rationale.

Constant	Value	Reason
F_CPU	8 MHz	Convenient timer values, low 9600-baud error, sufficient processing speed.
USART_BAUD_UBRR	51	Gives about 9615 baud in normal asynchronous mode.
OCR0 for ms delay	124	125 counts at 8 μ s per count equals 1 ms.
OCR0 for us delay	7	Eight no-prescaler counts at 0.125 μ s per count equals 1 μ s.
ICR1	19999	20,000 one-microsecond counts produce a 20 ms servo frame.
Servo locked pulse	1000 counts	Approximately 1 ms; mapped to 0 degrees.
Servo open pulse	1500 counts	Approximately 1.5 ms; mapped to 90 degrees.
OCR2	77	78 Timer2 counts with /1024 produce approximately 9.984 ms.
POWER_CONFIRM_TICKS_10MS	10	About 100 ms stability confirmation.
MAX_FAIL_ATTEMPTS	3	Project requirement for security lock-out.
LOCKOUT_SECONDS	30	Project requirement.
SLEEP_TIMEOUT_SEC	10	Approximately ten seconds of inactivity, as specified.
SERVO_OPEN_SEC	5	Project requirement.
TAMPER_THRESHOLD	50 counts	Approximately 0.244 V at a 5 V ADC reference; large enough to reject minor noise.
TAMPER_UI_HOLD_SEC	5	Blocks keypad and sleep while the remote tamper alarm is active.
FAIL_UI_HOLD_SEC	1	Brief input lock after an ordinary wrong password.

Constant	Value	Reason
REMOTE_ALARM_SEC	5	Project requirement for ordinary Slave alarm messages.
BREACH_ALARM_SEC	30	Matches the Master lockout duration.
SUCCESS_LED_SEC	5	Matches the Master servo-open interval.
RX_LINE_MAX	79	Large enough for log lines while limiting SRAM use.
LOG_MAX_RECORDS	63	63 records times 8 bytes plus an 8-byte header exactly fills 512 bytes.

Chapter 10

Proteus Test Procedure and Expected Results

10.1 Basic startup test

Both MCU clock properties are set to 8 MHz. The Master should show Enter Password;; the Slave should show REMOTE ALARM and WAITING.... The servo should be at the locked position. No remote alarm output should be active.

10.2 Correct password

Enter 1, 2, 3, and 4. The Master should show four asterisks, send LOGIN_OK, and move the servo to approximately 90 degrees. The Slave should show the access-granted message and turn on the green LED. After approximately five seconds, the servo returns to 0 degrees and the green LED turns off.

10.3 Incorrect password and breach

Enter an incorrect four-digit code. The Slave should show the wrong-password message and activate the red alarm for five seconds. After the third failure, the Master must reject input for thirty seconds, while the Slave displays the breach message and keeps the red alarm active for thirty seconds. After the lockout, the Master sends LOCKOUT_END, the Slave returns to waiting, and the Master accepts keypad input again.

10.4 Tamper test

With the Master awake, move the PA3 potentiometer far enough that the ADC difference exceeds 50 counts. The Master sends Tamper and stores the event. The Slave displays the tamper message and activates its red alarm for five seconds. The Master LCD remains the password interface. The tamper system rearms after the sensor becomes stable.

10.5 Power-failure test

Begin with PB3/AIN1 above the bandgap threshold. Lower the monitored voltage below the divider trip point and keep it there for more than 100 ms. The Master sends and logs `POWER_FAIL`, turns off its LCD, disables nonessential peripherals, and enters comparator standby. The Slave displays the persistent power alarm. Raising the voltage above the threshold wakes the Master, which reinitializes its peripherals, sends `POWER RESTORED`, and returns to the password prompt.

Pressing `RESET` while the voltage remains low should not restore normal operation. After the bandgap stabilization delay, the startup comparator state is recognized as failed and the Master returns to standby.

10.6 Inactivity sleep

Leave the Master inactive for ten seconds while no lockout, servo interval, or UI hold is active. The Master LCD turns off and the MCU enters Power-down. USART and sensor changes do not wake it. Pressing the `INT0` button wakes it and restores the password prompt.

10.7 Log dump

Wake the Master, connect the Master TX line to the virtual-terminal RX line, and press `D`. The terminal displays the log between the start and end markers. The Slave ignores all lines in this section. Pressing `C` clears the log and returns a confirmation. Terminal commands reset the inactivity timer so that the Master remains awake after the operation.

Chapter 11

Limitations and Possible Improvements

1. The system timestamp stops during Power-down and during power-failure standby because Timer2 is stopped. A real-time clock would provide wall-clock timestamps across sleep and reset.
2. The 25LC040 logger writes bytes individually. A page-write implementation would reduce write time and EEPROM command overhead.
3. The log uses a simple XOR checksum. A stronger CRC would provide improved corruption detection.
4. The comparator threshold has no hardware hysteresis. The 100-ms filter reduces chatter, but a resistor feedback network or a supervisor IC would provide a cleaner threshold in physical hardware.
5. The simulated power failure does not remove VCC from the MCU. A real last-gasp design would require rail isolation, energy storage, and a voltage supervisor.
6. The USART protocol is plain text and has no checksum. A framed binary protocol with sequence numbers could improve reliability.
7. A passive speaker requires PWM tone generation and a transistor driver. An active buzzer is simpler for the present on/off alarm logic.
8. Password change is supported by the EEPROM module but is not exposed through the final user interface.
9. The internal EEPROM failure count causes write wear if a very large number of failures is generated. Wear-leveling is unnecessary for the course simulation but may be considered in a product design.

Chapter 12

Conclusion

The completed design satisfies the central requirements of a two-controller smart safe. The Master performs local access control, sensing, logging, power management, and communication. The Slave provides remote, message-driven indication without sharing any sensor or actuator control with the Master. Timer and baud calculations are derived directly from the 8 MHz clock, and register settings are selected to produce stable LCD timing, standard servo PWM, periodic scheduling, reliable ADC conversion, SPI EEPROM access, and asynchronous serial communication.

The project also demonstrates several important embedded-system practices: separating interrupt and main-loop responsibilities, preserving timer state around blocking delays, handling write-one-to-clear flags correctly, performing atomic reads of 32-bit values on an 8-bit MCU, validating persistent memory, filtering noisy comparator transitions, prioritizing persistent alarms, and designing explicit low-power state transitions. The Proteus simulation provides a practical demonstration of the interaction between digital peripherals, analog sensing, nonvolatile storage, and communication in a complete security application.

Appendix A

Master Main-Loop Pseudocode

```
1 initialize all Master modules
2 if watchdog reset occurred:
3     send and log WDT_RESET
4 enable watchdog
5 clear password input
6 request LCD redraw
7
8 forever:
9     reset watchdog
10
11     process filtered comparator state
12     if a power event is pending:
13         if failure:
14             send complete POWER_FAIL
15             log power failure
16             enter comparator-only standby
17             reinitialize after restoration
18         else:
19             send and log POWER RESTORED
20
21     if 100 ms flag is set:
22         run tamper detection
23         if a new event occurs:
24             send and log Tamper
25             start five-second UI hold
26
27     service servo, lockout, and UI-hold expiration
28     process D/C terminal commands
29
30     if ten-second sleep conditions are satisfied:
31         enter Power-down with INT0 wake
32
33     redraw Enter Password prompt when requested
34
35     if keypad input is blocked:
36         continue
37
```

```
38     scan keypad
39     process C or a digit
40     after four digits:
41         check password
42         perform success, failure, or breach sequence
```

Appendix B

Slave Main-Loop Pseudocode

```
1 initialize Slave LCD, USART, Timer2, LEDs, and alarm
2 show REMOTE ALARM / WAITING
3 enable watchdog
4
5 forever:
6     reset watchdog
7     turn off temporary indicators whose counters reached zero
8
9     if no USART byte is available:
10         continue
11
12     read one byte
13     if byte is CR or LF:
14         terminate the line
15         if line is valid:
16             if log-dump start marker:
17                 enter ignore mode
18             else if in ignore mode:
19                 leave ignore mode only at end marker
20             else:
21                 compare exact event messages
22                 update LCD and output state
23         reset receive buffer
24     else:
25         append byte unless the line overflowed
```

Appendix C

Selected Register Setup Listings

C.1 USART

```
1 void usart_init(uint16_t ubrr)
2 {
3     UBRRH = (uint8_t)(ubrr >> 8);
4     UBRRL = (uint8_t)ubrr;
5     UCSRB = (1U << RXEN) | (1U << TXEN);
6     UCSRC = (1U << URSEL) |
7             (1U << UCSZ1) | (1U << UCSZ0);
8 }
```

C.2 Timer1 servo PWM

```
1 TCCR1A = (1U << COM1A1) | (1U << WGM11);
2 TCCR1B = (1U << WGM13) | (1U << WGM12) |
3         (1U << CS11);
4 ICR1   = 19999U;
5 OCR1A  = 1000U;
```

C.3 Timer2 scheduler

```
1 TCCR2 = 0;
2 TCNT2 = 0;
3 OCR2  = 77U;
4 TIFR  = (1U << OCF2);
5 TIMSK |= (1U << OCIE2);
6 TCCR2 = (1U << WGM21) |
7         (1U << CS22) | (1U << CS21) | (1U << CS20);
```

C.4 ADC3

```
1 DDRA  &= ~(1U << PA3);
2 PORTA &= ~(1U << PA3);
3 ADMUX = (1U << REFS0);
4 ADCSRA = (1U << ADEN) |
5           (1U << ADPS2) | (1U << ADPS1);
```

C.5 SPI

```
1 DDRB |= (1U << PB4) | (1U << PB5) | (1U << PB7);
2 DDRB &= ~(1U << PB6);
3 PORTB |= (1U << PB4);
4 SPCR = (1U << SPE) | (1U << MSTR) | (1U << SPR0);
5 SPSR = 0;
```

C.6 Analog comparator

```
1 DDRB  &= ~(1U << PB3);
2 PORTB &= ~(1U << PB3);
3 SFIOR &= ~(1U << ACME);      /* use dedicated AIN1 */
4 ACSR = (1U << ACBG);         /* bandgap, interrupt off */
5 delay_ms(1);                 /* reference stabilization */
6 initial_state = (ACSR & (1U << AC0)) ? 1U : 0U;
7 ACSR = (1U << ACBG) | (1U << ACI);
8 ACSR = (1U << ACBG) | (1U << ACIE);
```

Bibliography

- [1] Course Project Specification, *SmartSafe: Smart Safe with Remote Alert*, Microprocessors End-Term Project, Dr. Raji, Academic Year 1404.
- [2] Microchip Technology Inc., *ATmega32A Data Sheet*, DS40002072A. <https://ww1.microchip.com/downloads/en/DeviceDoc/Atmega32A-DataSheet-Complete-DS40002072A.pdf>
- [3] Microchip Technology Inc., *25AA040A/25LC040A 4-Kbit SPI Bus Serial EEPROM Data Sheet*, 20001827J. <https://ww1.microchip.com/downloads/en/DeviceDoc/25AA040A-25LC040A4-Kbit-SPI-Bus-Serial-EEPROM-20001827J.pdf>
- [4] Hitachi, *HD44780U Dot Matrix Liquid Crystal Display Controller/Driver Data Sheet*. <https://cdn.sparkfun.com/assets/9/5/f/7/b/HD44780.pdf>